

Si vous êtes débutants,
ou si votre binôme travail
sur les machines Polytech :
regardez plutôt la solution
"Polytech" même dans
votre propre machine

PHP et MySQL Installation si vous avez votre propre machine

[Ceci n'est pas le cours, mais important à le faire pour développer ensuite.]

PHP est un langage de programmation qui tourne côté serveur, qui génère des pages web dynamiquement. Mais pour le faire tourner nous avons besoin d'un serveur web. Mais ceci est un peu complexe. D'abord il faut avoir accès à une machine "visible" dans le web pour installer votre serveur, qui implique à la fois des questions de stockage et de sécurité (en particulier quand on apprend :/). Et en plus c'est difficile à déboguer car votre navigateur ne voit finalement qu'une page web en html (et pas le code qui l'a généré).

La solution proposée ici est si vous avez votre propre machine et que vous êtes admins.

- Il y a un tuto video en web qui est *presque* complet si vous voulez suivre un vidéo mais attention, son

Docker container n'est pas complet et la base de données n'est pas la même que la vôtre. [https://](https://www.youtube.com/watch?v=2Bxh5FNGznQ)

www.youtube.com/watch?v=2Bxh5FNGznQ

1. Serveur Web, installation Docker

Pour ce cours nous allons installer un serveur web local qui tourne, mais nous allons le faire à travers de Docker. Docker est une plateforme de virtualisation au niveau du système d'exploitation pour fournir des logiciels dans des paquets appelés "containers". L'idée (on simplifie) est que le "container" tourne une machine virtuelle, et donc nous pouvons utiliser et tourner ce container de la même façon peu importe la platform et machine qu'on utilise.

Il y a quelques étapes à faire avant de commencer :

1. Installer **Docker Desktop** (qui installe aussi le service Docker Engine) <https://docs.docker.com/desktop/>. Cela nécessite de créer un compte (free).
2. S'assurer que votre Docker Desktop tourne (et donc que le Docker Deamon tourne).
3. Dans votre Visual Studio Code (VSC) ajoutez l'extension Docker (de Microsoft) et PHP Debug et PHP IntelliSense pour aider avec le débogage et auto-completion.
4. Dans Visual Studio Code vous pouvez créer un dossier pour travailler. Dedans vous pouvez ajouter un fichier php (ex, `docker-test.php`) qui contient le code

```
<?php
    echo "hello!";
    phpinfo();
?>
```

4. Pour l'instant nous ne pouvons pas interpréter ce fichier, nous avons besoin de tourner un serveur web. Nous allons donc créer un fichier `docker-compose.yml` avec Visual Studio Code (dans le même dossier que le fichier `php`). Ce fichier définit les services que notre machine virtuelle ("container") doit tourner. Vous pouvez ajouter le code suivant:

```
services:
  www:
    image: php:apache
    volumes:
      - "./:/var/www/html" # synch current project dir with web dir
    ports:
      - 80:80
      - 443:433 # for future ssl traffic
```

Un peu d'explication pour Docker :

- `www` indique que nous allons ajouter un service d'un serveur web qui tourne *apache* (serveur web classique) et `php` - ceci est indiqué dans `image`.
- `volumes` sont nos "disks" ou stockage du serveur. Nous disons que l'endroit ".", c'est à dire le dossier actuel ou existe le fichier `docker-compose.yml`, devrait jouer le rôle (mapped ":") du dossier avec les pages web (en serveurs linux il s'agit de `/var/www/html`).
- `ports` sont le porte du machine virtuelle et son mapping vers les portes de notre machine (80 est réservé pour des services web).
- le `#` est un commentaire

2. Tourner le Serveur

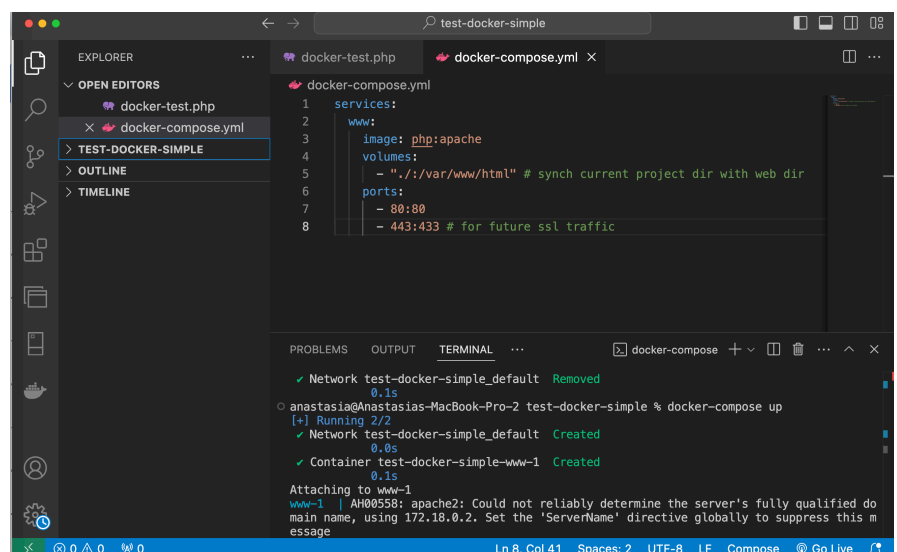
Nous sommes prêts à tourner le serveur Docker (d'abord vérifiez que votre Docker Desktop tourne, càd le Docker Deamon tourne).

Nous pouvons tourner notre Docker container à partir de VSC :

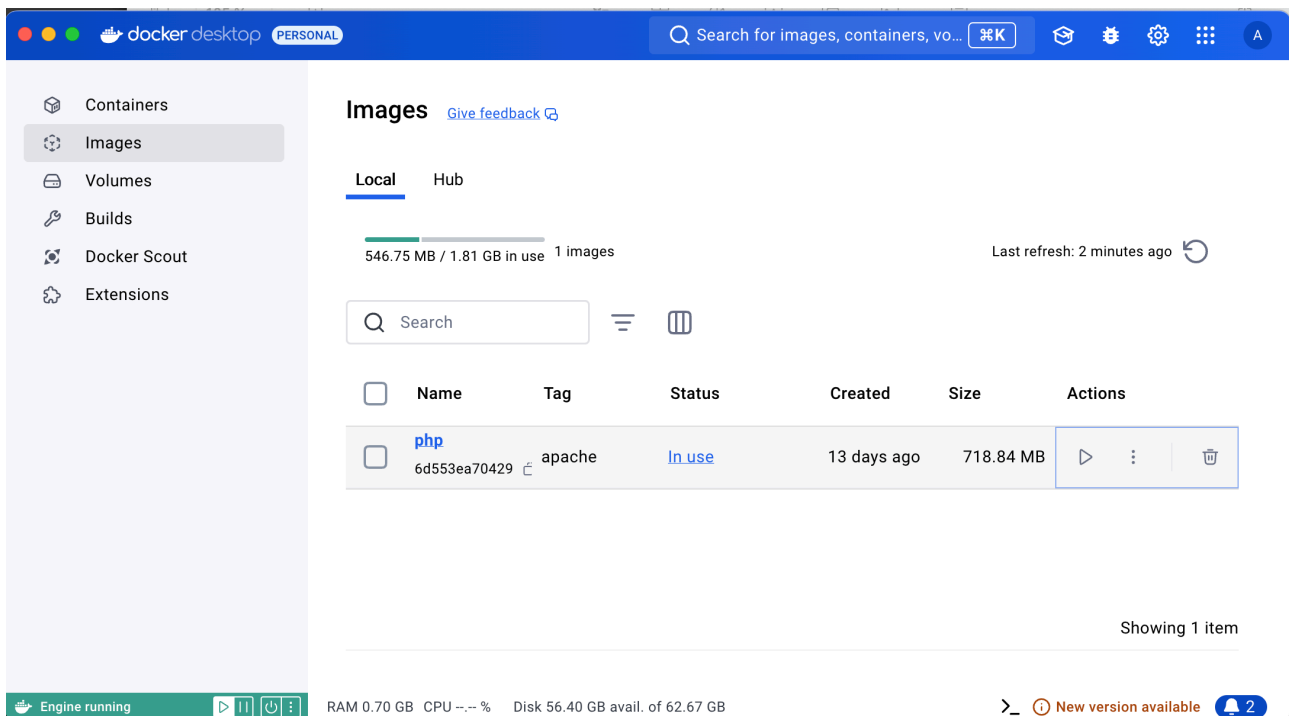
1. Il faut ouvrir son Terminal (CTRL ~)
2. Ecrire dans le Terminal : **docker-compose up**

Cela commence votre Docker et lit le fichier de configuration (`docker-compose.yml`)

Voici une capture d'écran de VSC qui montre ça.

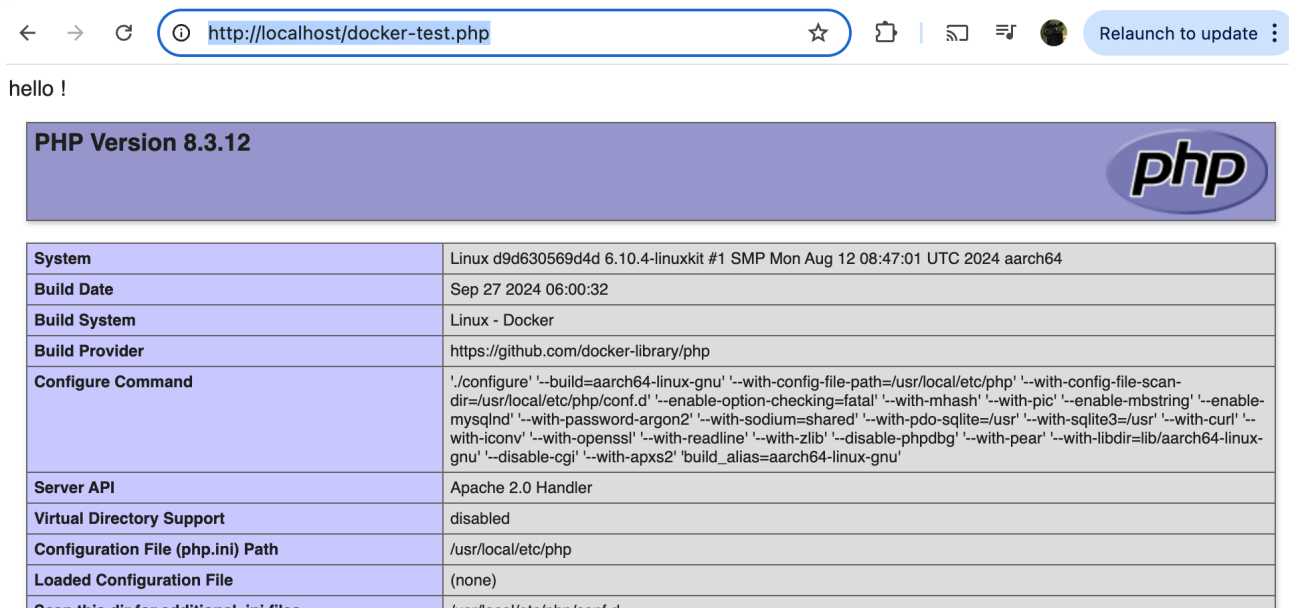


Et si vous regardez dans votre Docker Desktop vous pouvez voir votre service Apache / Php (donc votre serveur web) qui tourne, comme dans l'image ici.



Nous avons un serveur qui tourne !

Si vous visitez dans votre navigateur le site <http://localhost/docker-test.php> vous pouvez voir la page docker-test.php maintenant interprété par le serveur et rendu comme une page html dans votre navigateur.



Quand vous avez fini votre travail vous pouvez écrire dans le Terminal de VSC (CTRL C) pour mettre Docker en pause, puis **docker-compose down** pour terminer le service. Ou vous pouvez le terminer dans Docker Desktop.

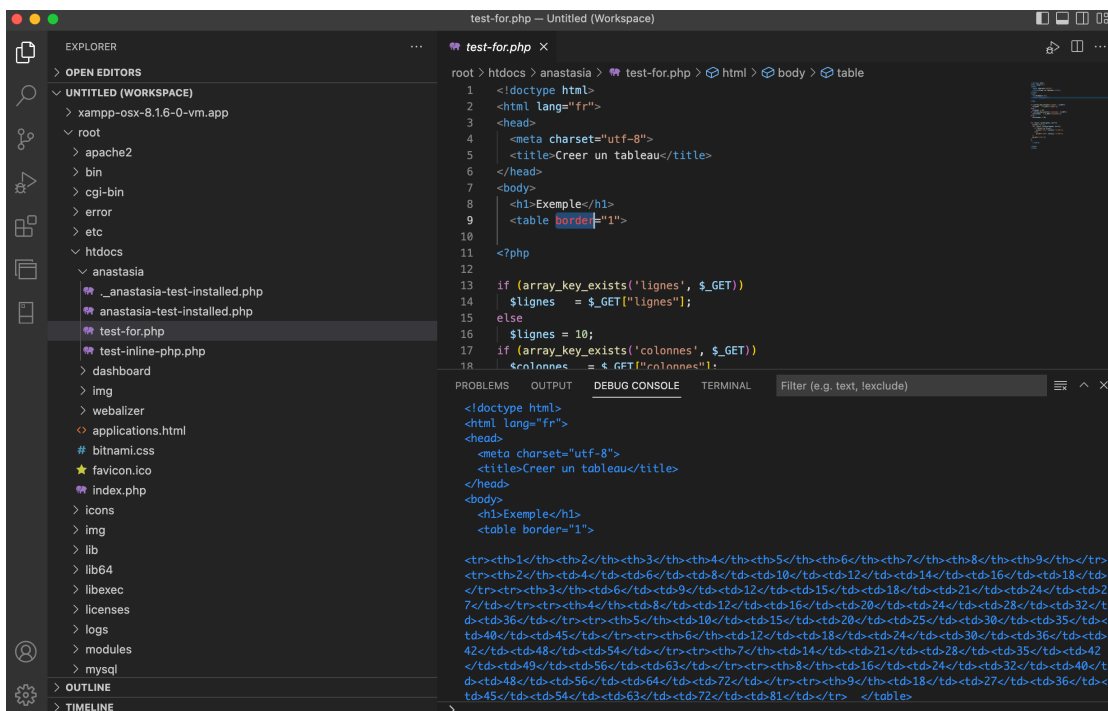
3. Environment de développement PHP

Comme PHP est sensé tourner dans le serveur web, notre navigateur reçoit finalement une page web en html et donc nous ne pouvons pas utiliser "inspect" pour déboguer notre code PHP.

Il y a plusieurs environnements de développements, autrement dites IDEs pour développer PHP [IDE = Integrated Development Environment]. Exemples sont Eclipse ou NetBeans (qu'on utilise souvent pour Java), Atom avec des extensions pour PHP, etc. Si vous avez une préférence allez-y !

Un IDE intéressant est Visual Studio (si vous êtes en Windows), ou **Visual Studio Code** (pour MacOS et Linux) <https://visualstudio.microsoft.com/free-developer-offers/>. Une version de Visual Studio existe dans les machines de l'école. En plus, la version Visual Studio Code ne nécessite pas d'installation avec droits d'administrateur, donc vous pouvez la télécharger et cela doit marcher dans les machines de l'école aussi.

Voici une image de mon installation. Vous voyez le code, des problèmes de syntax (sous Problems) et dans le Debug Console on voit le code html généré par le code PHP.



Vous pouvez :

- (i) ajouter des extensions  pour PHP : comme PHP Debug, PHP Extension Pack, PHP IntelliSense, PHP Intelephense).
- (ii) ajouter des extensions  pour MySQL plus tard.
- (iii) ouvrir votre "Application Folder / Volume" dans Explore  pour voir vos fichiers.

À partir d'ici on a des infos sur l'installation MySQL, nécessaire pour cours en SQL

4. Installation MySQL

Dans le dossier ou existent le reste de vos fichiers ajoutez un dossier db, ici nous allons stocker notre base de données.

Pour ajouter de MySQL, nous allons ajouter dans notre docker compose des services liés aux BD. Donc nous pouvons éditer notre `docker-compose.yml` avec VSC pour avoir cette version :

```
services:
  www:
    image: php:apache
    command: "/bin/sh -c 'docker-php-ext-install mysqli && exec apache2-foreground'" # install mysqli to communicate with DB
    volumes:
      - "./:/var/www/html" # synch current project dir with web dir
    ports:
      - 80:80
      - 443:433 # for future ssl traffic
  db:
    image: mysql:latest # service for mySQL
    environment:
      - MYSQL_DATABASE=appinfo # database for our php test
      - MYSQL_USER=php_docker # user name for the DB
      - MYSQL_PASSWORD=password # not a great password ...
      - MYSQL_ALLOW_EMPTY_PASSWORD=1 # set to true to have users without passed
    volumes:
      - "./db:/docker-entrypoint-initdb.d" #sync persistent sql files with container because Aiiiii Docker does not save the DB
  phpmyadmin:
    image: phpmyadmin/phpmyadmin # have a service for an interface of the DB
    ports:
      - 8001:80 # define a port to be able to see the BD
    environment:
      - PMA_POST=db # which DB service to connect to
      - PMA_PORT=3306 # mySQL uses this port
```

Aiiiii, bon on a ajouté pas mal de choses sur la configuration de docker (pour éviter de faire des Dockerfile pour tous les services). Vous pouvez lire les commentaires pour comprendre un peu les différent composants / services mais en sommaire :

- bd : une base des données (bd : MySQL), nous définissons le nom de la BD et un compte pour se connecter.
- phpmyadmin : phpMyAdmin est un logiciel libre écrit en PHP, destiné à gérer l'administration de MySQL sur le Web.
- nous avons aussi ajouter dans `www` une commande pour installer `mysqli` (on parlera de ça dans le cours SQL).

Attention: le service de Docker n'est pas persistant, càd si on l'arrête nous allons perdre notre BD c'est pourquoi si vous utilisez docker, à la fin de la session vous devrez faire une copie de votre base des données et l'ajouter dans le dossier db.